Name of Student: VIKAS FOOLCHAND KUSHWAHA			
Roll Number: 29/B		**Lab Practical Number:** 06	
Title of LAB Practical: Implementation of Server, Piping and Routing.			
DOP: 08/11/2026		**DOS:** 12/11/2026	
CO Mapped: CO1	**PO Mapped:** PO3, PO5, PSO1, PSO2	**Faculty Signature:**	**Marks:**

Aim: Implementation of Server, Piping and Routing.

Description:

HTTP server

An HTTP server receives requests from clients—usually web browsers—and sends back responses. In Node.js, the built-in http module lets you create a server and handle each request with a callback.

A request contains information such as the URL, HTTP method (GET, POST, etc.), headers, and possibly a body. The server uses that information to decide what to do. A response includes a status code, headers, and a body. For example, 200 indicates success, while 404 means the requested resource was not found. The Content-Type response header tells the browser what kind of content it received, such as HTML, JSON, or PDF.

Piping

Piping connects a readable stream to a writable stream. For a file download, the file stream is the source and the HTTP response is the destination. Data passes through in chunks instead of requiring the server to load the entire file into memory first.

This is useful for large files such as videos. It can reduce memory use and let the client receive data as it becomes available. Streams also manage backpressure: if the destination cannot accept data as quickly as the source produces it, the flow is regulated. For video playback, the server can additionally support byte-range requests so a browser can request parts of the file, which helps with seeking.

Routing

Routing is the process of matching an incoming request to the code or resource that should handle it. A route is commonly determined by the URL path and HTTP method. For example, a GET request to /about might return an About page, while a GET request to /contact returns a Contact page.

If no route matches, the server should respond appropriately, usually with 404 Not Found. Separating route handling from page or business logic makes an application easier to understand and maintain.

Express

Express is a lightweight web framework for Node.js. Node's built-in HTTP module provides the low-level tools to create a server; Express adds convenient abstractions for common web-development tasks, including defining routes, using middleware, serving static files, and handling errors.

Middleware is code that runs while a request is being handled. It can perform tasks such as parsing request data, logging requests, checking authentication, or serving static files. Express is deliberately flexible, so developers choose how to organize routes and which additional libraries to use.

Install Express in order to setup a server.

```
> npm install express

added 65 packages in 3s

28 packages are looking for funding
  run `npm fund` for details
```

Code:

QUESTION 1: **Create a HTTP Server and serve HTML, CSV, JSON and PDF Files.**

File: file-server.js

```
const express = require('express');
```

```
const fs = require('fs');
const path = require('path');

const app = express();
const port = 3000;

// Home Page
app.get('/', (_, res) => {
  res.sendFile(path.join(__dirname, 'assets', 'index.html'));
});

// Serve HTML files
app.get('/html', (_, res) => {
  res.sendFile(path.join(__dirname, 'assets', 'students.html'));
});

// Serve CSV files
app.get('/csv', (_, res) => {
  res.sendFile(path.join(__dirname, 'assets', 'students.csv'))
});

// Serve JSON files
app.get('/json', (_, res) => {
  res.sendFile(path.join(__dirname, 'assets', 'students.json'))
});

// Serve PDF files
app.get('/pdf', (_, res) => {
  res.sendFile(path.join(__dirname, 'assets', 'students.pdf'));
})

app.listen(port, () => {
  console.log(`Server is running on https://localhost:${port}`)
});
```

QUESTION 2: **Create a HTTP Server and stream a video file using piping.**

File: video-stream.js

```
const http = require('http');
```

```
const fs = require('fs');
const path = require('path');

const port = 3000;

const server = http.createServer((req, res) => {
  if (req.url === '/video') {
            const   videoPath   =   path.join(__dirname,   'movies',
'shortfilm.mp4');
    console.log(videoPath);
    const stat = fs.statSync(videoPath);
    const fileSize = stat.size;
    const range = req.headers.range;
    if (range) {
      const parts = range.replace(/bytes=/, '').split('-');
      const start = parseInt(parts[0], 10);
      const end = parts[1] ? parseInt(parts[1], 10) : fileSize - 1;

      const chunkSize = end - start + 1;
      const file = fs.createReadStream(videoPath, { start, end });
      const head = {
        'Content-Range': `bytes start −{end}/${fileSize}`,
        'Accept-Ranges': 'bytes',
        'Content-Length': chunkSize,
        'Content-Type': 'video/mp4',
      };
    } else {
      const head = {
        'Content-Length': fileSize,
        'Content-Type': 'video/mp4',
      }
      res.writeHead(200, head);
      fs.createReadStream(videoPath).pipe(res);
    };
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('Not Found');
  };
});

server.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`)
```

```
});
```

QUESTION 3: **Develop 3 HTML Web Pages for college home page (write about college & dept), about us and contact. Create a server and render these pages using Routing.**

File: server.js

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const server = http.createServer((req, res) => {
  let filePath = '';

  switch (req.url) {
    case '/':
    case '/home':
      filePath = path.join(__dirname, 'public', 'home.html');
      break;
    case '/about':
      filePath = path.join(__dirname, 'public', 'about.html');
      break;
    case '/contact':
      filePath = path.join(__dirname, 'public', 'contact.html')
      break;
    default:
      res.writeHead(404, { 'Content-Type': 'text/html' });
      res.end('<h1>404 - Page Not Found</h1>');
      return;
  }

  fs.readFile(filePath, (err, content) => {
    if (err) {
      res.writeHead(500, { 'Content-Type': 'text/plain' });
      res.end('Server Error');
    } else {
      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.end(content);
    }
  })
```

```
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
})
```

File: public/home.html

```
<!doctype html>
<html lang="en">

  <head>
    <meta charset="utf-8">
    <title>Home Page</title>
  </head>

  <body>
    <nav>
      <a href="/">Home</a> |
      <a href="/about">About Us</a> |
      <a href="/contact">Contact</a>
    </nav>

    <h1>VES Institute of Technology</h1>
    <p>
          Welcome to Vivekanand Education Society's Institute of Technology. We are pleased to see you here. Go ahead and visit our other pages on this website or directly contact us via the contact link.
    </p>

    <h2>Computer Science Department</h2>
    <p>
        This section is for our computer science department. We are thrilled to have you here exploring computers and computer technology. It is one of the fastest growing fields in the Global industry. We hope you will join us in this new era of technological revolution.
    </p>
  </body>
```

```
</html>
```

File: public/about.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>About us</title>
</head>

<body>
  <h1>About Us</h1>
  <p>
    We are the members of VESIT.
  </p>
</body>

</html>
```

File: public/contact.html

```
<!DOCTYPE html>
<html lang="en">

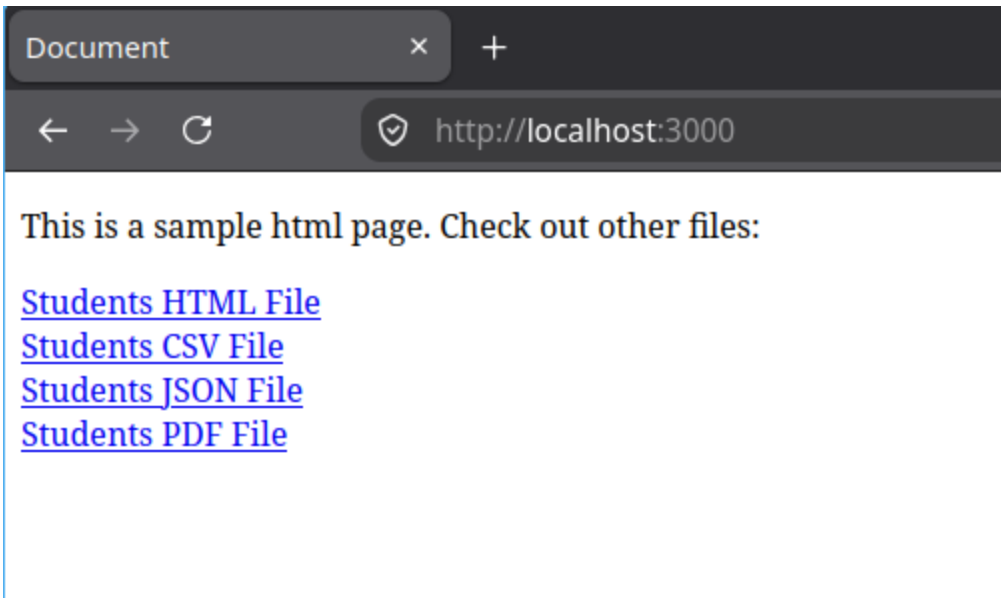
<head>
  <meta charset="UTF-8">
  <title>Contact US</title>
</head>

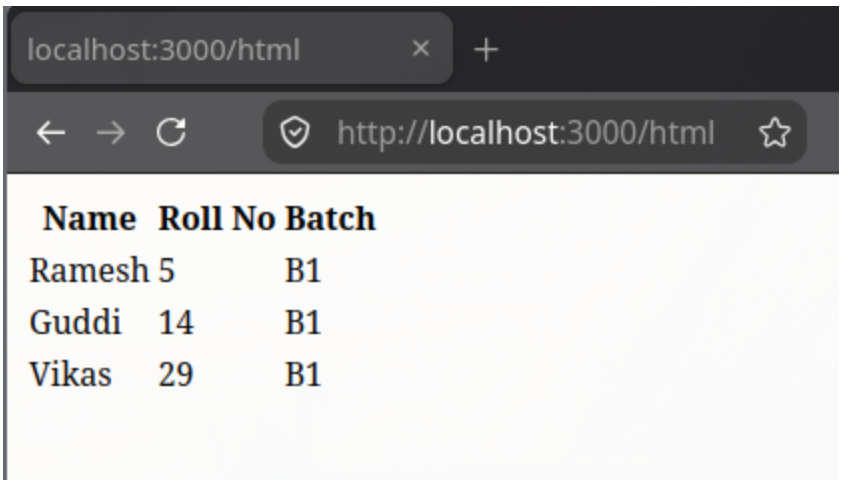
<body>
  <h1>Contact Us</h1>
  <p>
    We are located in Collector Colony, Chembur, Mumbai.
  </p>
</body>
```

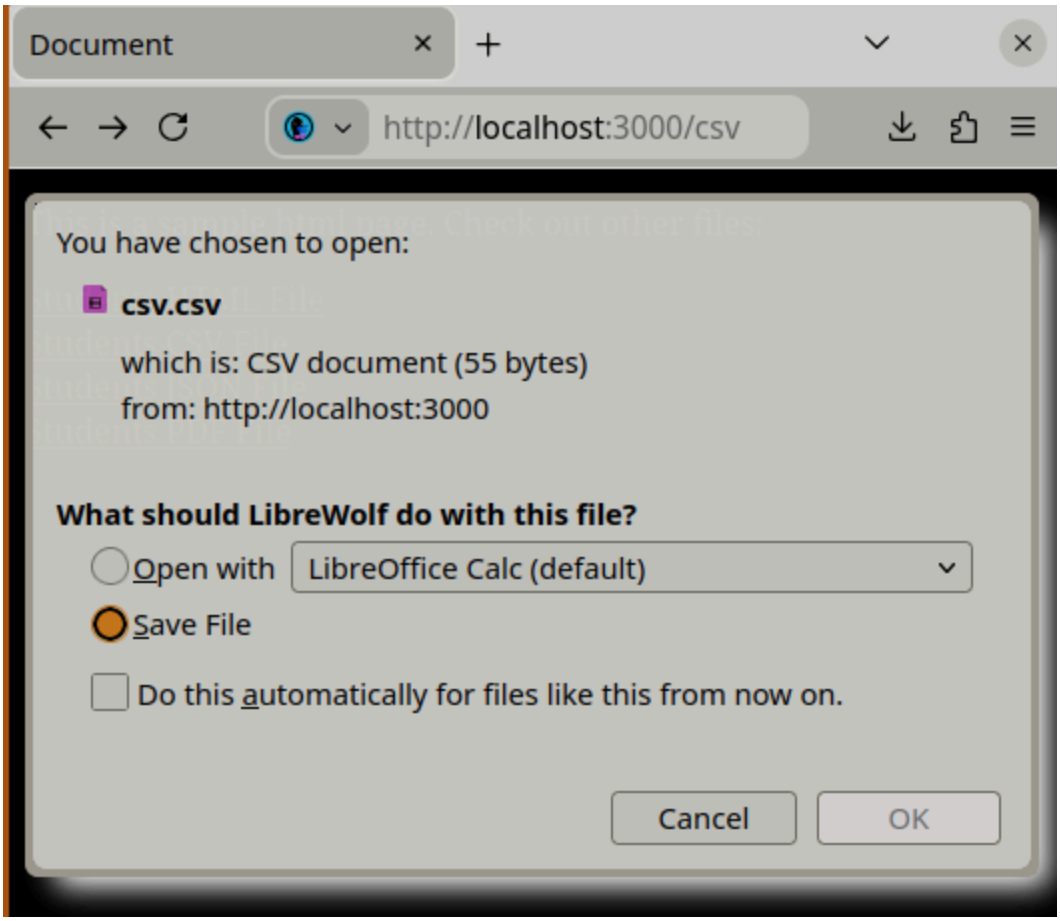
```
</html>
```

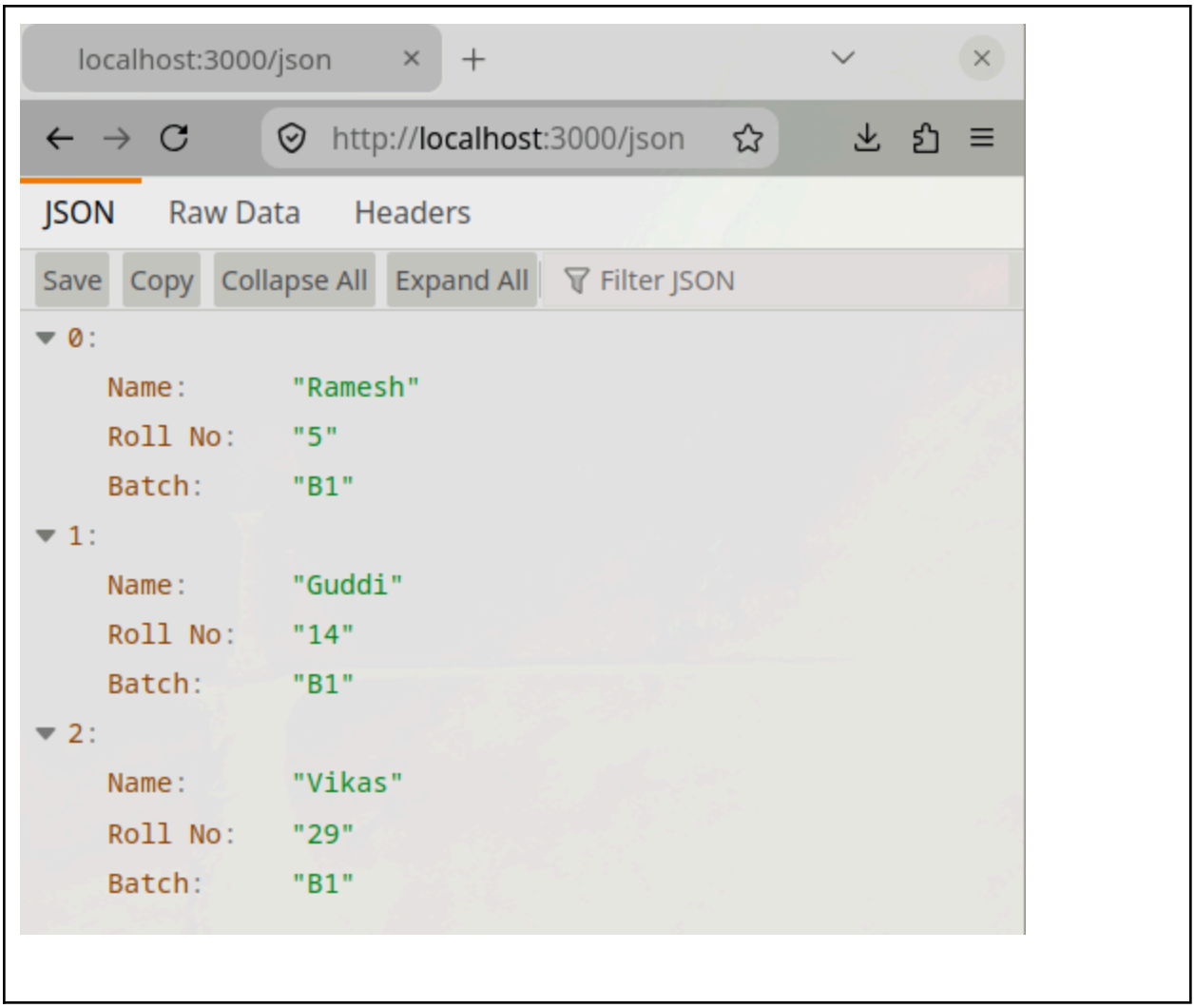
Output:

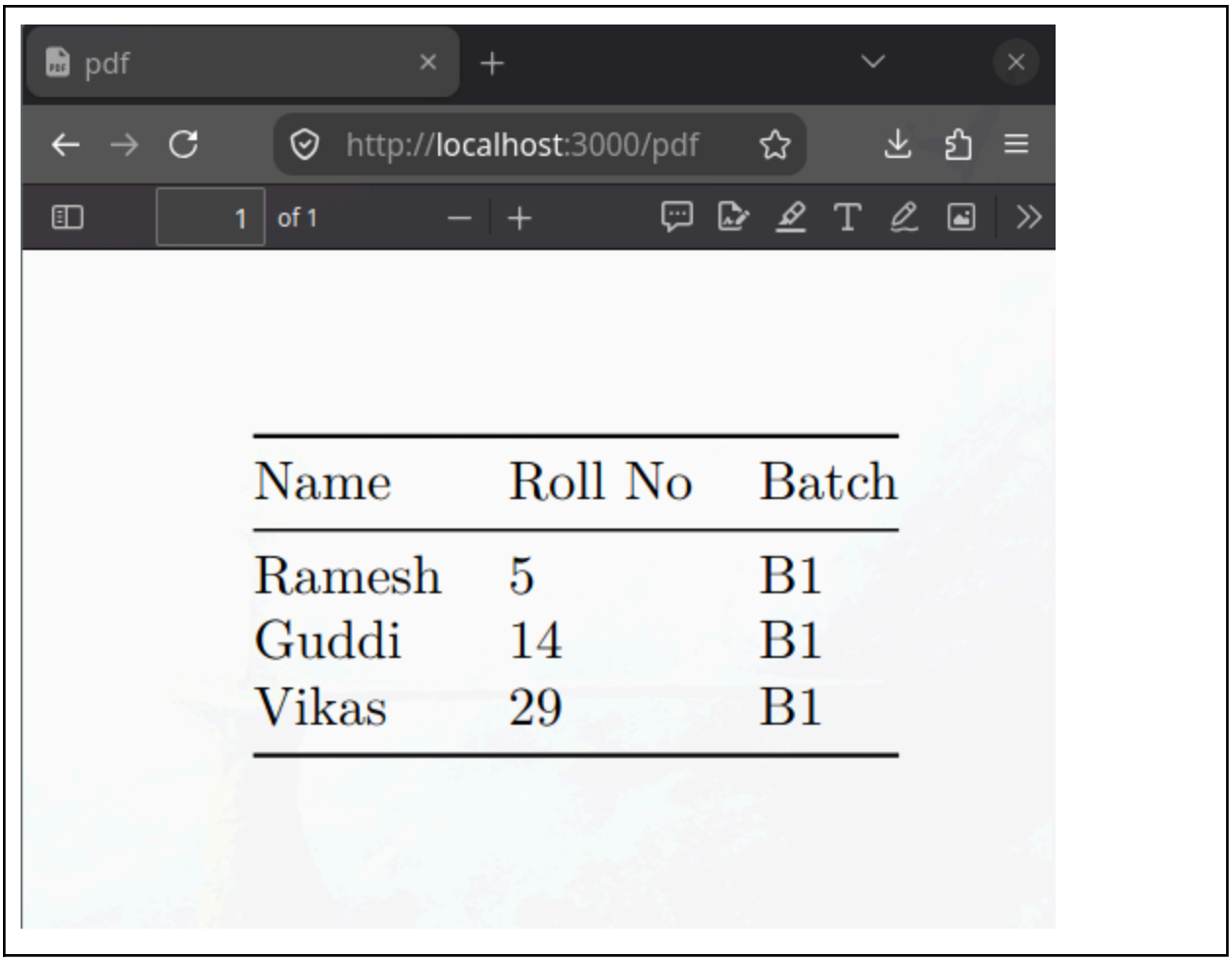
OUTPUT 1:




localhost:3000/html
http://localhost:3000/html
Name Roll No Batch
Ramesh 5 B1
Guddi 14 B1
Vikas 29 B1


Document
http://localhost:3000/csv
You have chosen to open:
csv.csv
which is: CSV document (55 bytes)
from: http://localhost:3000
What should LibreWolf do with this file?
Open with
LibreOffice Calc (default)
Save File
Do this automatically for files like this from now on.
Cancel
OK


localhost:3000/json
http://localhost:3000/json
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
0:
Name: "Ramesh"
Roll No: "5"
Batch: "B1"
1:
Name: "Guddi"
Roll No: "14"
Batch: "B1"
2:
Name: "Vikas"
Roll No: "29"
Batch: "B1"



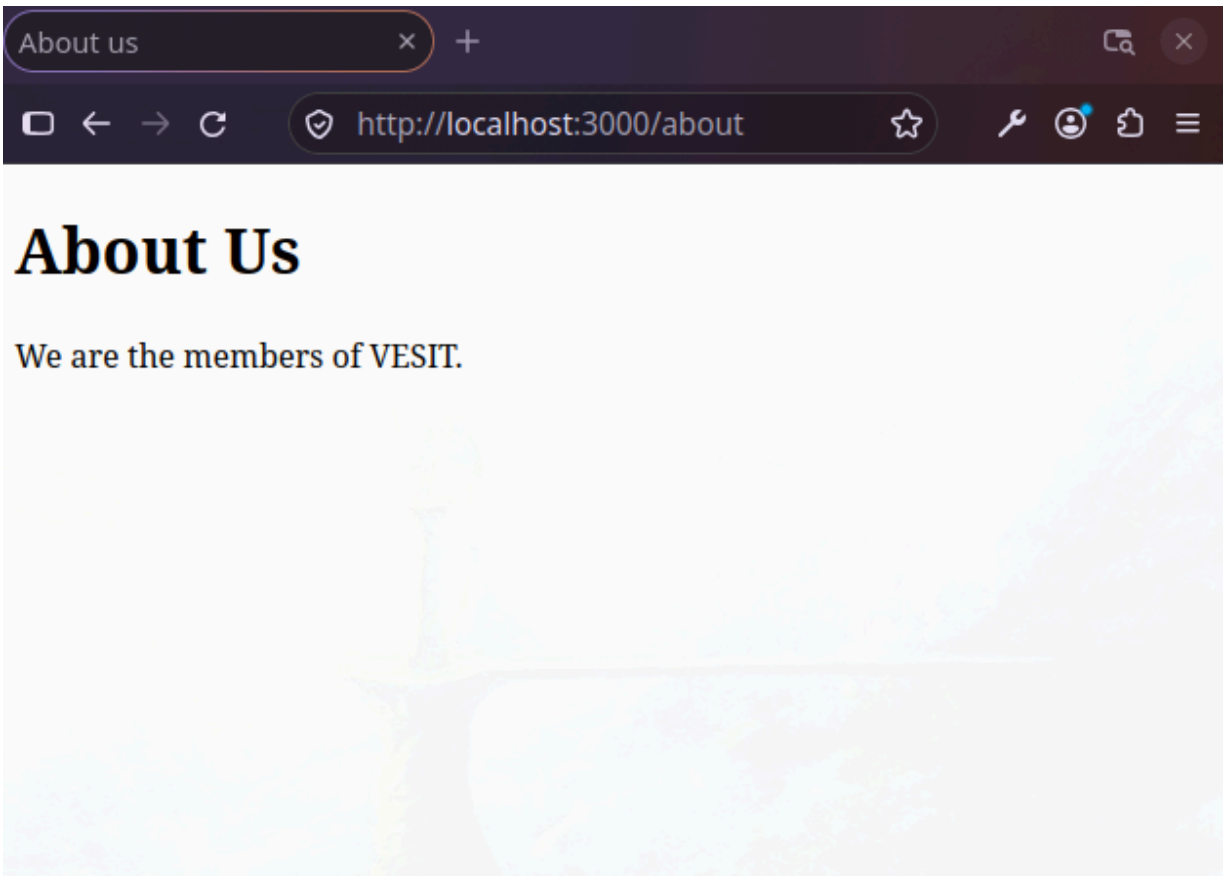
Name	Roll No	Batch
Ramesh	5	B1
Guddi	14	B1
Vikas	29	B1

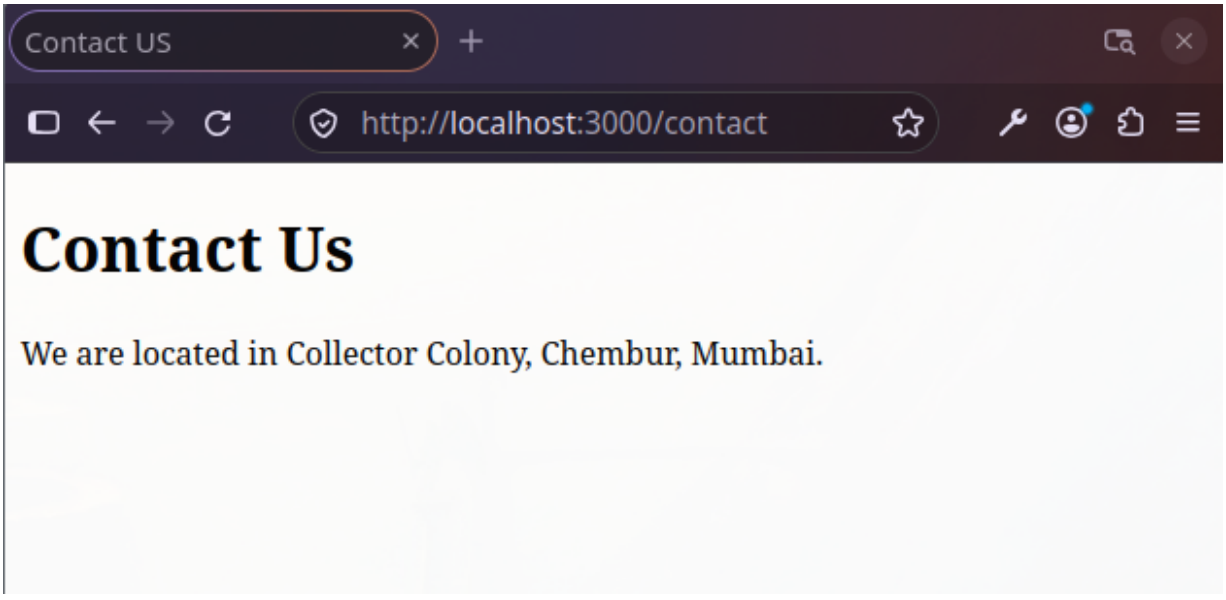
OUTPUT 2:



OUTPUT 3:


Home Page
http://localhost:3000
Home | About Us | Contact
VES Institute of Technology
Welcome to Vivekanand Education Society's Institute of Technology. We are pleased to see you here. Go ahead and visit our other pages on this website or directly contact us via the contact link.
Computer Science Department
This section is for our computer science department. We are thrilled to have you here exploring about computers and computer technology. It is one of the fastest growing fields in the Global industry. We hope you will join us in this new era of technological revolution.


About us
http://localhost:3000/about
About Us
We are the members of VESIT.


Contact US
http://localhost:3000/contact
Contact Us
We are located in Collector Colony, Chembur, Mumbai.

Conclusion:

In this practical, we implemented web servers using nodejs ‘http’ module and the ‘express’ web framework. We handled client requests, streamed video files through piping and served various static web pages using routing. This exercise demonstrates the following key concepts:

1. **HTTP Server:** We learned how to set up a web server that listens on a specific port and handles client requests and serves content.

2. **Piping:** We efficiently streamed a large video file by breaking it into chunks and sending them to the client one by one, also showcasing memory efficient data handling.

3. **Routing:** We implemented various routes for handling different kinds of URLs and served multiple html files dynamically.